

Facultade de Informática
UNIVERSIDADE DA CORUÑA

TRABALLO FIN DE GRAO
GRAO EN ENXEÑARÍA INFORMÁTICA
MENCIÓN EN TECNOLOXÍAS DA INFORMACIÓN

Análise, deseño e implementación dunha gramola virtual.

Estudante: Carlos Mañoso Sierra
Dirección: Diego Andrade Canosa

A Coruña, September de 2021.

Para todas as persoas que desexan escoitar a súa canción favorita.

Agradecementos

A todos eses que desexaban ver este proxecto rematado, especialmente a Cris e a Sergio.

Resumo

En calquera situación na que se escoite música, sempre hai alguén que prefire un estilo, unha canción, ou unha lista de reprodución ao completo. Para resolver este problema na década dos 60 comezaron a introducirse as gramolas, deste xeito arranxabase o problema, pero eran aparellos grandes e nos que tan só se podían escoitar os éxitos do momento e nos lugares nos que existían estes reprodutores. Hoxe en día a maneira de escoitar música sufriu un cambio grande, a maioría reprodúcese a través de Internet, dende plataformas coma pode ser Spotify. Isto aumentou a variedade musical e simplificou o poder escoitar música en calquera lugar, é estraño unha festa con amigos sen música, polo tanto son frecuentes as persoas que piden escoitar a súa canción favorita.
O obxectivo deste proxecto é resolver este problema recorrendo a mesma idea que antano, actualizado a forma de escoitar música hoxe en día. Crearase unha aplicación web cun backend en Django Rest Framework capaz de manexar pola súa conta as cancións na cola de reprodución e un frontend en Ionic con Angular para a iteración cos usuarios dende a web.
Obtivemos unha aplicación que xestiona unha lista de reprodución compartida, capaz de reproducila en calquera aparello que incorpore Spotify coma a súa plataforma de música. Nesta, dunha maneira moi sinxela os usuarios poderán crear unha sesión para compartir, conectala co seu Spotify e deixar que o resto dos usuarios engadan as cancións que desexen e voten polas que mais lles gustan.

Abstract

In any situation where music is listened to, there is always someone who prefers a style, a song or a compleat playlist. To solve this kind of problem in the 60s the jukeboxes appeared, this way the problem was solved, but the machines were big, you could listen just the hits of the moment and where the device were. Nowadays how we listen to music changed a lot, usually the music is played throght the Internet, from platforms like Spotify. This increased the music variety and made easier to listen music anywhere, it's weird a party with friends without music, so is usual the people that asks to listen their favourite song.
The objetive of the project it's to solve this problem the same way yesteryear, updated to the way we listen music nowadays. We will build a web application with a Django Rest Framework backend able to manage the songs at the queue and a frontend in Ionic with Angular, so the users can interact with the web.
We got an app that manages a shared playlist, able of play it at any device with Spotify as

music platform. In a very simple way the users will be able to create a session to share, connect it with their Spotify and let the other users add songs or vote for their favourite ones.

Palabras chave:

- Gramola
- Música
- Bar
- Cancións
- Spotify
- Web app
- Ionic
- Django

Keywords:

- Jukebox
- Music
- Bar
- Songs
- Spotify
- Web app
- Ionic
- Django

Índice Xeral

1 Introdución **1**

1.1 Obxectivos . . . 2

1.2 Estrutura da memoria . . . 2

2 Análise **5**

2.1 Viabilidade económica . . . 5

2.2 Viabilidade técnica . . . 6

2.3 Viabilidade do mercado . . . 6

2.4 Análise . . . 7

2.5 Requisitos da aplicación . . . 8

2.5.1 Requisitos funcionais . . . 8

2.5.2 Requisitos non funcionais . . . 8

2.5.3 Actores . . . 8

2.6 Metodoloxía . . . 9

2.7 Scrum . . . 9

2.7.1 Roles . . . 9

2.7.2 Reunións . . . 10

2.7.3 Artefactos Scrum . . . 10

2.7.4 Scrum no proxecto . . . 10

3 Tecnoloxías **13**

3.1 Ferramentas de desenvolvemento . . . 13

3.2 Frameworks e librarías . . . 14

3.3 Linguaxes . . . 15

3.4 Sistema de xestión de base de datos . . . 17

3.5 APIs . . . 17

3.6 Estado da arte . . . 18

4 Planificación **25**
4.1 Planificación inicial . . . 25
4.2 Adaptación da planificación a Scrum . . . 25
4.2.1 Primeiro sprint . . . 26
4.2.2 Segundo sprint . . . 26
4.2.3 Terceiro sprint . . . 26
4.2.4 Cuarto sprint . . . 26
4.2.5 Quinto sprint . . . 26
4.3 Custos do proxecto . . . 27

5 Deseño **29**
5.1 Lóxica de funcionamento . . . 29
5.2 Arquitectura . . . 30
5.2.1 MVC: Modelo, vista, controlador . . . 30
5.2.2 MVT: Modelo, vista, plantilla . . . 30
5.2.3 Esquema xeral . . . 31
5.3 Frontend . . . 32
5.3.1 Arquitectura . . . 32
5.3.2 Vista . . . 32
5.3.3 Controller . . . 33
5.4 Backend . . . 36
5.4.1 Modelo . . . 36
5.4.2 Plantilla . . . 37
5.4.3 Vista . . . 37

6 Conclusións e traballo futuro **43**
6.1 Conclusións . . . 43
6.2 Traballo futuro . . . 43

Bibliografía **49**

Índice de Figuras

3.1 TouchTunes 18
3.2 Ami Entertainment 19
3.3 Jukestar App 20
3.4 Outloud App 20
3.5 Captura dunha sala en Jukebox Today 21
3.6 Raveberry Server 22

4.1 Mockups do frontend 27

5.1 Model, Template, View. 31
5.2 Estrutura da base de datos 37

Índice de Táboas

2.1 Custos dos plans de Spotify 6

4.1 Tempo empregado no proxecto 27

Capítulo 1

Introdución

NESTE primeiro capítulo da memoria, se explica á motivación, se expoñen os obxectivos e se define á estrutura.

Durante a décadas dos 60 e 70 a industria musical sufriu unha revolución, nunha gran parte dos lugares de restauración comezaron aparecer aparellos que permitían elixir as seguintes cancións no local, as gramolas. Coa dixitalización da música, as gramolas foron desaparecendo e en poucos casos adaptándose os novos formatos e dixitalizandose. Nos nosos días, coas múltiples plataformas que existen e a música na nube é moi complicado atopar un bar no que se permita elixir a seguinte canción, coma antes. Do mesmo xeito en calquera lugar onde soe música sempre hai alguén que quere engadir a súa canción a cola de reprodución, este pode ser un obxectivo sinxelo ou mais complicado dependendo de moitos factores. Con este proxecto tratamos de resolver estes problemas e dun xeito doado compartir os diferentes gustos musicais.

O obxectivo deste proxecto é crear unha aplicación web na que os usuarios poidan engadir as cancións que desexen a unha lista de reprodución compartida e votar polas súas favoritas. Deste xeito as cancións reprodúcense segundo a orde de chegada e segundo o seu número de votos. Polo tanto a idea é crear unha gramola virtual na que os usuarios non teñen que recorrer a ningún aparello, con calquera dispositivo con acceso a Interet deberían de ser capaces de conectarse a gramola e engadir cancións. Para realizar a reprodución de música usase o cliente de Spotify que se controla mediante a súa propia API deste xeito se xestiona a música que vai soar, polo tanto poderá ser reproducida en calquera dispositivo que teña un cliente desta plataforma. Evítase interromper o uso normal do Spotify, é dicir, facendo que o usuario poida usar o seu cliente de Spotify coma sempre e facendo que a aplicación engada as cancións a cola de reprodución tan so nos momentos necesarios. Co propósito de facilitar o seu uso e desta maneira incrementar o número de usuarios creouse a plataforma do xeito mais intuitivo posible para estes.

Na parte técnica o obxectivo é crear una Progressive web application (PWA), é dicir unha

aplicación creada coas tecnoloxías web, que funciona en calqueira tipo de navegador compatible. A aplicación constará dun backend en Django Rest Framework, que será o encargado de xestionar as solicitudes a API de Spotify, e manexar as propias peticións do noso frontend en Ionic que utiliza Angular coma framework, en canto a base de datos utilizaremos SQLite para o almaceamento dos usuarios e as listas de reprodución.

1.1 Obxectivos

O proxecto pretende crear unha plataforma de creación de listas de reprodución de música de forma colaborativa dunha forma similar a unha gramola e a reprodución destas mediante un cliente de Spotify. A continuación expoñense os obxectivos funcionais:

- Xestión de sesións nas que os administradores poderán conectar a súa conta de Spotify para a reprodución de música.
- Poderán crearse novos usuarios dentro de cada unha das sesións, contando estas cun usuario administrador e os demáis usuarios rexistrados.
- Posibilidade de facer unha búsqueda de cancións para engadir a cola de reprodución.
- As cancións na cola de reprodución poderán votarse para cambiar a orde de reprodución destas.
- O administrador tan só necesitará conectar a súa conta de Spotify e abrir un cliente deste, non será necesario ter a aplicación aberta nun navegador.

Do mesmo xeito expoñemos os obxetivos non funcionais:

- As comunicacións tanto coa API propia coma coa de Spotify deberán de ser seguras.
- A usabilidade do sistema ten que ser sinxela, tanto o rexistro coma a búsqueda de música ten que ser clara.
- Funcionamento sen manter a aplicación aberta por parte do administrador.
- A versión web deberá ser responsible, adaptada tanto a pantallas grandes coma a móbiles.

1.2 Estrutura da memoria

Este documento estruturase en 7 capítulos.

- Neste primeiro capítulo describese a motivación, e os obxectivos do proxecto.

- No capítulo 3.
- No capítulo terceiro explícanse as tecnoloxías utilizadas durante o proceso de desenvolvemento.
- No capítulo 4.
- No capítulo 5.
- No capítulo 6.
- No capítulo 7.

Capítulo 2

Análise

NESTE segundo capítulo se fará unha análise da viabilidade do proxecto, deste xeito poderemos determinar se é posible que chegue a finalizarse con éxito. Posteriormente expoñeremos a metodoloxía de traballo utilizada para o desenvolvemento do proxecto.

Para determinar se o proxecto poderá acadar o seu obxetivo existen varios aspectos a analizar, o conxunto destes é a viabilidade do proxecto. En xeral para os proxectos software as áreas nas que recopilar información para determinar a viabilidade son: técnica, económica e de mercado, deste xeito pódese saber de meneira anticipada cal vai ser o custo aproximado do proxeto, o tempo que vai levar rematalo e cal é o impacto no mercado esperado, a continuación descríbense estes diferentes dominios.

2.1 Viabilidade económica

No análise económico teñense en conta tres custos principais, os custos en recursos humanos, en software e en hardware.

Para os **recursos humanos**, téñense en conta a cantidade de persoas necesarias e o custo por hora de cada unha destas. Neste caso é unha soa persoa a encargada de facer o deseño, implementación e probas, tampouco é necesario ter en conta o custo por hora polo tanto non existe un custo real monetario en recursos humanos para a realización do proxeto.

Os **recursos software** teñen en conta os custos do software utilizado para o desenvolvemento do proxecto. Neste proxecto utilizase software libre e gratuíto na maior medida posible, partindo do sistema operativo ata os frameworks e librarías, pasando polas ferramentas de desenvolvemento coma o editor de texto ou o navegador coma veremos mais adiante 3.1, todo isto fai que os custos software neste proxecto sexan nulos, a excepción da necesidade de unha subscripción ao servizo *premium* de Spotify. Na táboa ?? podemos observar os diferentes prezos do servizo, neste caso o plan elexido é o familiar polo tanto o custo mensual ascende a 2,67€ mensuais e aproximadamente de 14€ durante a realización do proxecto.

Táboa 2.1: Custos dos plans de Spotify

Nome	Individual	Duo	Familiar	Estudantes
Número de contas	1	2	6	1
Prezo mensual	9,99€	12,99€	15,99€	4,99€
Prezo por conta	9,99€	6,50€	2,67€	4,99€

En canto aos **recursos hardware**, non foi necesario adquirir ningún tipo de equipo, xa que para o desenvolvemento o único necesario é un ordenador con conexión a Internet. Sopesouse a idea de necesitar un servidor na nube para despregar a aplicación o completo, pero debido ao pequeno tamaño da aplicación poderíase recurrir a un servizo gratuíto se fose necesario.

Polo tanto dedúcese que respecto a viabilidade económica este proxecto é completamente factible xa que os custos vense moi reducidos incluso os necesarios son asumibles xa que o servizo *premium* de Spotify é algo que ten mais utilidades que o desenvolvemento.

2.2 Viabilidade técnica

A viabilidade técnica é a referente a o funcionamento do sistema, condiciona se as características tecnolóxicas permiten que o proxecto acade un fin exitoso. Neste caso o software escollido para o desenvolvemento ainda que sendo relativamente novo é altamente coñecido e empregado na creación de proxectos similares, además se dispón de moita documentación e moi completa, incluso tutoriais específicos, o que facilita a realización do proxecto polo tanto non presenta unha dificultade técnica alta. Xuntando estes motivos, afírmase que o proxecto é viable tecnicamente.

2.3 Viabilidade do mercado

No apartado do estado da arte 3.6 verase que existen diferentes compañías que ofrecen software e hardware comercial que comercializan gramolas virtuais físicas para establecementos onde se pode engadir música directamente dende un aparello, estas compañías atópanse algo implantadas en Norte América e comezan a popularizarse algo en Europa. En canto a software libre non existen opcións comerciais que cumpran este fin dun xeito similar, e o que existe é tedioso de instalar e mais complexo para o administrador. Tan só hai algúns proxectos ou aplicacións móbil que de forma xeral crean unha gramola virtual de maneira local para que os usuarios poidan engadir cancións, incluso Spotify ten unha función de crear unha sesión para compartir con catro usuarios mais, pero que non é tan doado de utilizar en espazos mais grandes, xa que están plantexados para utilizar en pequenas reunións ou lugares con poucos

usuarios. Non se atopou ningún proxecto que permita crear unha gramola virtual sen manter outra aplicación aberta e distribuído de forma gratuíta. Por estes motivos podemos afirmar que este proxecto ten cabida no mercado, entón podemos consideralo coma viable.

Poñendo en conxunto os análises de viabilidade pódese resumir que o proxecto ten cabida no mercado, é viable economicamente e técnicamente. Polo tanto dedúcese que o proxecto é viable e é probable que teña éxito no mercado.

2.4 Análise

Para a creación deste proxecto se van desenvolver un frontend baseado en Ionic no que se amosarán todos os datos relacionados coas cancións que están na cola, o número de votos e a súa orde de reprodución. Tamén se vai realizar un backend en Django Rest Framework que nos permitirá a conexión do frontend coa API de Spotify e así levar a lóxica do proxecto a cabo. As funcións destacadas son as seguintes:

- Creación dunha sesión.
  Poderase crear unha sesión onde se vai reproducir a música, será necesario facer a autenticación contra a API de Spotify cunha conta premium. Esta sesión pode interpretarse como sería o bar ou lugar de emprazamento dunha gramola física, pero por suposto que pode ser usado en oficinas, reunións particulares e calquera sitio onde a música teña cabida.

- Creación de usuarios.
  Os usuarios poderán rexistrarse nas sesións, o primeiro en facelo terá o rol de administrador, deste xeito terán a posibilidade de engadir cancións a cola de reprodución e votar a orde destas.

- Creación dunha cola de reprodución.
  Os usuarios poderán engadir cancións a unha cola de reprodución xenérica para a sesión, a aplicación encargarase de que cando remate unha canción comece a reprodución da seguinte.

- Posibilidade de facer votacións.
  Na cola de reprodución aparecen o número de votos de cada canción e botóns que sirven para engadir ou substraer votos, a lista de reprodución estará ordenada por este número de votos e posteriormente por orden de chegada, coma sería nunha gramola habitual.

- Poboar a cola de reprodución cunha lista xa creada.
  Aproveitando o potencial das listas de reprodución que ten Spotify, a cola de reprodu-

ción poderá ser enchida cunha playlist do usuario, o obxectivo de esta funcionalidade é que os usuarios poidan votar a orde de reprodución das cancións da lista.

2.5 Requisitos da aplicación

Os requisitos son as necesidades funcionais que ten un produto para satisfacer o seu obxectivo. Os requisitos poden ser funcionais ou non funcionais, os primeiros son os que describen o comportamento de funcións particulares no software, os segundos non están relacionados especificamente coa funcionalidade do sistema.

2.5.1 Requisitos funcionais

- Amosar información da cola de reprodución.
- Amosar os resultados de procura.
- Engadir cancións a cola.
- Permitir a creación de sesións.
- Permitir as votacións aos usuarios.
- Permitir facer unha procura de cancións.
- Permitir o rexistro de usuarios.
- Facer votacións positivas ou negativas.

2.5.2 Requisitos non funcionais

- **Deseño:** minimalista fácil de comprender e sinxela de usar.
- **Consistencia:** O modelo da aplicación mantense actualizado sen perder datos.
- **Seguridade:** Tan só os usuarios rexistrados poden facer uso da aplicación.
- **Fiabilidade:** Os erros en produción están controlados para poder dar unha resposta ó usuario en caso de que se dean.

2.5.3 Actores

No proxecto existen tres tipos de usuarios, usuarios sen rexistrar, usuarios rexistrados e administrador.

- **Usuario sen rexistrar:** O usuario sen rexistrar ten acceso a páxina de inicio dende onde se pode crear unha nova sesión ou facer login.
- **Usuario rexistrado:** Os usuarios rexistrados poden engadir ver a cola de reprodución, buscar e engadir cancións e votar positiva ou negativamente as cancións.
- **Usuario administrador:** O usuario administrador pode facer todo o que os usuarios rexistrados e ademais pode poboar a cola de reprodución con listas de reprodución de Spotify, tamén é o usuario que engade o token a sesión.

2.6 Metodoloxía

A forma de traballo consistirá no estudo dos proxectos relacionados, para continuar coa análise, deseño e implementación e probas do proxecto. Utilizarase a metodoloxía Scrum para o desenvolvemento.

2.7 Scrum

Scrum Scrum é un proceso de desenvolvemento áxil. Permite un desenvolvemento rápido e por iteracións, permítenos inspeccionar o software creado e cambiar as preferencias de traballo no lugar dunha planificación e execución completa dende o principio. É un método no que se aplican de maneira regular un conxunto de boas prácticas para traballar de forma colaborativa e obter o mellor resultado posible. Os equipos organizasen entre eles para discutir as funcionalidades e a prioridade destas, deste xeito despois de cada iteración se pode facer unha demostración do produto nese momento.

2.7.1 Roles

Os roles da metodoloxía son os seguintes:

- Product Owner: é o encargado de fixar as metas do sprint e a priorización do Product Backlog. O seu obxectivo é que o produto maximice o seu valor.
- Scrum Master: a súa responsabilidade é promover as prácticas Scrum, media entre o equipo de desenvolvemento e o Product Owner. Dirixe as reunións diarias e as revisións dos Sprints.
- Equipo de desenvolvemento: organizanse para desenvolver o proxecto seguindo o Sprint Backlog.

2.7.2 Reunións

- Daily Scrum: tratase dunha reunión diaria duns 15 minutos de duración que fai o equipo de desenvolvemento, é dirixida polo Scrum Master e nela cada participante responde a 3 sinxelas preguntas. 'Qué fixeches onte?´, 'Qué vas facer hoxe?´, 'Qué problemas atopaches?´, deste xeito pódese ver o estado actual do proxecto.

- Sprint Planning: nesta reunión se seleccionan as tarefas a realizar durante o Sprint, se fai unha estimación da súa duración e se obtén o Sprint Backlog, participan tanto o equipo de desenvolvemento coma o Scrum Master.

- Sprint Review: ao final de cada Sprint realizase esta reunión que ten como obxectivo presentar os traballos realizados neste, poden realizarse adaptacións do Product Backlog.

- Sprint Retrospective: tamén se realiza ao final de cada sprint e ten como finalidade analizar o traballo realizado, as funcionalidades e as melloras.

Unha vez rematadas tanto o Sprint Review e o Sprint Retrospective iniciase a planificación do seguinte Sprint Planning de forma cíclica.

2.7.3 Artefactos Scrum

- Product Backlog: é a lista que recolle todos os requisitos do produto, ordenado de maior a menor prioridade. Trátase dunha lista dinámica xa que vai cambiando cos diferentes Sprints.

- Sprint Backlog: agrupa en tarefas os elementos do Product Backlog seleccionados para o Sprint actual.

2.7.4 Scrum no proxecto

Neste proxecto, debido as diferencias coa realidade se fixeron modificacións da metodoloxía. Coma non hai un equipo de desenvolvemento, os roles de Scrum Master e o equipo de desenvolvemento foron manexados polo alumno e o rol de Product Owner polo director.
Cada Sprint Planning tivo unha duración de aproximadamente dúas semanas, o Daily Scrum foi substituído por un repaso das tarefas realizadas e un diario de traballo e o Sprint Planning realizado de forma individual.
Para a creación do Product Backlog se realizaron reunións ao principio do desenvolvemento para a concreción dos aspectos fundamentais da aplicación, foi cambiando co paso do tempo segundo as versións funcionais do produto.

Os Sprint Backlog foron seleccionados polo alumno segundo as prioridades establecidas inicialmente, ademais de seguir unha orde lóxica de desenvolvemento de funcionalidades, dende as mais básicas ata as específicas.
Tanto o Sprint Review coma o Sprint Rertospective reducidas debido a complexidade.

Capítulo 3

Tecnoloxías

NESTE terceiro capítulo explicase as tecnoloxías utilizadas durante o proceso de desenvolvemento, as linguaxes usadas no proxecto xunto coas librarías, os frameworks e utilidades externas, por último explicarase o estado da arte respecto a nosa idea.

3.1 Ferramentas de desenvolvemento

Para a creación deste proxecto utilizouse unha estrutura de desenvolvemento sinxela realizada na maior parte nun terminal Linux xa que este é moi versátil e lixeiro, a continuación describense as ferramentas que foron utilizadas e comentando as súas características principais.

Comezando polo sistema operativo, utilizouse **Linux** todo o código do proxecto foi desenvolvido en máquinas con *Arch Linux*, unha distribución moi lixeira acompañada dunha gran cantidade de aplicacións instalables diretametne co seu xestor de paquetes *Pacman*, tamén é unha das primeiras en recibir actualizacións de paquetes coma o *Kernel*, ademais conta con unha enorme comunidade que permite documentarse e resolver problemas rapidamente.

Para e edición do código escolleuse **NeoVim** trátase un editor de texto na terminal, creado para aumentar a eficiencia ao editar calquera tipo de texto. É un fork de *VIM*, e está deseñado do facilitar o seu mantemento, a súa extensibilidade e usabilidade, alenta aos seus usuarios engadiren contribucións. Aínda que non se trata dun Integrate Development Environment (IDE) grazas a comunidade existen múltiples complementos que fan o seu uso moi similar a un destes, pero sendo moito mais lixeiro en canto a recursos utilizados. Foi utilizado para o desenvolvemento do código e da memoria, aquí pódese atopar a configuración utilizada aínda que permanece en constante cambio.

Coma sistema de control de versións utilízase **Git** é un sistema distribuido de código aberto, deseñado para manexar todo tipo de proxectos, caracterízase principalmente pola súa velocidade, rendemento, usabilidade e flexibilidade. Facilita de gran maneira o rexistro de cambios

que se fan no código o que tamén permite o desenvolvemento de código de forma colaborativa.

Para o aloxamento do proxecto, facilitando así a súa compartición e a copia de seguridade utilizouse **GitHub**, que é unha plataforma de desenvolvemento colectivo que utiliza o sistema de control de versións de *Git*, utilizase principalmente para o control de versións e o almacenamento de código fonte.

No apartado de probas utilízase **Firefox developer edition** para comprobar o funcionamento da aplicación. Trátase dun dos navegadores mais utilizados, é de código libre e conta con unha comunidade que desenvolve unha gran cantidade de complementos. Nesta versión en concreto ponse énfase en facilitar o traballo aos desenvolvedores, xa que conta con características coma a visualización de cuadrículas CSS ou un depurador para *JavaScript*. Polo tanto pensouse que é o mais axeitado para o desenvolvemento de gran parte do frontend.

Para comprobar a compatibilidade tamén usáronse diferentes navegadores, **Firefox** na súa versión normal e na versión móbil, do mesmo modo **Google Chrome** probouse en ambas versións, na móbil e na de escritorio. Nos navegadores específicos dos sistemas *Windows* máis *OS X*, que son *Edge* e *Safari* respectivamente tamén foron utilizados mais sen facer probas exhaustivas.

No seu comezo era tan só unha extensión para o navegador, pero **Postman** agora é unha completa plataforma que permite a creación o uso de APIs de maneira mais sinxela, simplificando todos os pasos na creación destas, o que permite o seu desenvolvemento dunha forma máis rápida e mellor. No proxecto a utilizamos en lugar de HTTPie xa que se coñecía o seu funcionamento con anterioridade.

Para a realización dos mockups do frontend, utilizamos **Figma** un editor de gráficos vectoriais, tamén e unha ferramenta de xeración de prototipos baseada na web, aínda que conta con aplicacións para o escritorio.

Coa idea de controlar o tempo invertido e a xestión de tarefas utilizouse **Superproductivity** que é unha ferramenta de código libre deseñada precisamente para isto, permite levar un rexistro das tarefas e os tempos invertidos para cada unha de elas, ademais de contar con integración coas principais plataformas de desenvolvemento colectivo coma *Jira* ou *GitHub* o que permite a asignación de tarefas ou ¨tickets¨ de forma doada. Por desgraza non se utilizou durante todo o proxecto e tan só ao final polo que non foi posible afinar ao máximo os tempos empregados en total.

3.2 Frameworks e librarías

No desenvolvemento do proxecto utilizamos principalmente dous frameworks para o frontend e o backend e unha libraría que nos facilita as peticións a API de Spotify, a continuación describimolos.

Para o desenvolvemento do frontend empregamos **Ionic** un framework de código aberto que engloba un conxunto de ferramentas para o desenvolvemento de Progressive web application (PWA)[1], enfócase na experiencia do usuario e no interface, proporcionando múltiples controles para xestionar as iteracións. Unha das súas características mais chamativas é que permite co mesmo código a creación de aplicacións web, e aplicacións para os sistemas mais utilizados hoxe en día, *iOS* e *Android*. Escollimos esta plataforma debido a súa integración e a facilidade para desenvolver aplicacións web.

Ionic permite a integración cos frameworks *React*, *Vue* e *Angular* decantámonos por este último. **Angular** é un framework baseado en compoñentes para a xeración de páxinas web escalables, está desenvolvido en *TypeScript* e *HTML*, utiliza unha ampla gama de librarías que integran características coma as redirecións ou manexo de formularios. Decantámonos por este framework xa que é o que mais tempo leva funcionando en *Ionic* e contén características coma os observables que facilitan a xestión das peticións ao backend.

Na parte do backend eliximos **django**, trátase dun framework de código aberto escrito en *Python*. Na súa orixe foi creado para xestionar un xornal de noticias, facilita a creación de sitios webs complexos, pon énfase na reutilización da conectividade e a extensibilidade dos compoñentes e no desenvolvemento rápido. No noso caso utilizamos *Django Rest Framework*, que contén características mais específicas para a creación de APIs RESTful o que permite a transferencia de información entre as interfaces e as bases de datos, precisamente é o axeitado para o noso backend.

Por último, referente as librerías utilizamos **Spotipy** é unha libraría de Python que permite a execución das consultas na API de Spotify. Ademais tamén inclúe a xestión de *tokens* para gardar as credenciais de acceso dos usuarios, polo tanto dunha forma doada podemos interactuar coa plataforma de música. Tivemos en conta outras librerías coma Tekore pero a descartamos por non contar con tanta documentación e non satisfacer as necesidades do mesmo xeito.

3.3 Linguaxes

Sendo este un proxecto que utiliza un backend e un frontend con varios frameworks foi necesaria a utilización de varias linguaxes de programación.

Na parte do frontend para a visualización da aplicación utilizamos **HyperText Markup Language (HTML)** que é unha linguaxe de marcado, trátase dun estándar para documentos que se desexen ver nun navegador, utilízase na maioría das páxinas webs xa que define a estrutura básica destas. En conxunto con outras tecnoloxías coma Cascading Style Sheets (CSS) permite cambiar a aparencia ou con JavaScript podemos cambiar a súa funcionalidades

[1] Aplicacións, web progresivas

crear páxinas web dinámicas.

Para editar o aspecto das páxinas web utilizamos un procesador de CSS chamado **Sass**. É unha linguaxe de *scripting* que permítenos xerar de forma automática follas de estilo, engadindo características que non contén CSS e que son propias de linguaxes de programación coma poden ser as variables, funcións, selectores aniñados, herdanza... Isto axúdanos a manter unha organización modular dos estilos, ademais de que reduce o tempo de creación e mantemento do CSS.

Coma explicábamos con anterioridade, **JavaScript** serve para engadir melloras no interface e dinamizar páxinas web. É unha linguaxe de programación dinámica de scripting, orientada a obxectos, permite engadir contido dinámico e interactivo as páxinas web. Normalmente executase no propio navegador e pode ser inxerido en calquera parte do código HTML. Aínda que é amplamente utilizado na web non é exclusivo, utilízase tamén no desenvolvemento de xogos ou por exemplo *widgets*.

Debido a que *JavaScript* non pode detectar erros, utilizamos **TypeScript** que é un superconxundo de *JavaScript* deseñado para aplicacións extensas que a través dun compilador traducese a *JavaScript*, engade tipos estáticos e obxectos baseados en clases, principalmente é usado para o desenvolvemento de aplicacións do lado do cliente.

Na parte do backend utilizamos tan só **Python** xa que Django está escrito nesta linguaxe e todas as funcións que realizamos tamén. Trátase dunha linguaxe de programación interpretado, que fai fincapé na lexibilidade do código e utiliza a indentación para isto, do mesmo xeito tamén aposta pola modularidade e a reutilización do código, conta con un amplo repositorio de paquetes que estenden a súa funcionalidade de gran maneira. É unha linguaxe de alto nivel e multiparadigma xa que acepta tanto programación orientada a obxectos, imperativa e funcional. Tamén é un das linguaxes maior valuadas polos desenvolvedores.

Para o manexo da base de datos *Django* utiliza **SQL** que é unha linguaxe cun dominio específico, é usado para a administración, recuperación de bases de datos relacionais, particularmente útil usando modelos de datos. Unha das súas principais características é que usa álxebra relacional para efectuar as consultas, co fin de recuperar de forma sinxela información das bases de datos e para realizar cambios nela.

Por último, para a realización da memoria utilizamos **LaTeX** é un sistema de composición de textos de alta calidade. En xeral é usado para crear artigos e documentos técnicos e científicos aínda que pode utilizarse para todo tipo de textos. Segue a filosofía de separar a presentación e o contido, deste xeito os autores poden enfocarse no contido sen atender de forma simultanea a parte visual. Cando se prepara un documento o escritor especifica unha estrutura lóxica utilizando conceptos simples coma capítulo, sección ou táboa, e LaTeX encargase do formato, de todos modos permite a edición do estilo aínda que sexa dunha forma separada.

3.4 Sistema de xestión de base de datos

Xa sabemos que *Django* utiliza *SQL* para o manexo das bases de datos, pero é necesaria una implementación dun sistema xestor, no noso caso **SQLite**. É unha libraría escrita en *C* que implementa un motor de bases de datos SQL, lixeiro, rápido, autocontido e cunha implementación completa para SQL. É ideal para aplicacións que requiren unha instalación pequena e nas que a maioría das peticións son de tan só lectura. No noso proxecto o usamos para almacenar tanto os usuarios, as sesións e as cancións.

Debido a que *SQLite* está deseñado para bases de datos lixeiras non ten un gran soporte de concorrencia. O que quere dicir que non sempre vai ser capaz de xestionar os interbloqueos entre diferentes fíos de proceso, polo tanto é posible que teñamos erros de bloqueo da base de datos e non sexa posible acceder os datos requiridos en canto o número de usuarios da plataforma sexa moi grande. Por sorte Django permite cambiar o xestor de bases de datos e isto poderíase facer se comezásemos a sufrir estes problemas debido a alta demanda de usuarios e poderíamos poñer en funcionamento outro motor coma podería ser *mySQL* ou *MariaDB* xa que estes manexan de mellor xeito os bloqueos.

3.5 APIs

A base deste proxecto é reproducir música e para isto necesitábamos obter a música dende algún lugar. Nos primeiros momentos ponderamos o uso de varias plataformas de música conxuntamente coma poderían ser *YouTube*, *Deezer* ou *Apple Music* pero finalmente decantámonos por reproducir a música directamente na plataforma e facer modificacións na súa cola de reprodución o que facilitaría o funcionamento para os usuarios, para isto teríamos que escoller tan só unha. Escollemos *Spotify* xa que conta con una enorme biblioteca musical e tamén é unha plataforma que está moi afianzada, moitos locais tan só utilizan este servizo para toda a música que reproducen. Outra baza ao seu favor é a súa completa API que permítenos xestionar case todas as funcionalidades que poderíamos utilizar nun dos seus clientes, coma son a procura ou a xestión de listas de reprodución aínda que notamos en falta algunha funcionalidade coma é a consulta da cola de reprodución, xa que esta API tan só permite engadir cancións a esta e ningunha outra acción o que podería ternos facilitado o traballo. Por último tamén nos axudou a escoller esta plataforma a súa ampla documentación específica para a API que é moi detallada e conta con exemplos e incluso un cliente para facer probas.

Aínda que poderíase utilizar a **API de Spotify** directamente, nos utilizamos unha librería de *Python* chamada *Spotipy*, xa que implementa todas as funcionalidades da API e ademais facilita a obtención e gardado dos *tokens* dos usuarios, contando tamén con unha documentación extensa explicando cada unha das súas funcións.

3.6 Estado da arte

Na actualidade existen diferentes tipos de gramolas, en xeral todas son dixitais e obteñen a información a través de Internet. Hai compañías que ofertan o servizo de obter un aparello físico para dispoñer nun local, e que alí os clientes poidan dun xeito similar a antigamente facer a súa elección musical. Do mesmo xeito existen outras solucións que están baseadas en aplicacións, dun xeito similar ao noso proxecto, os clientes poden conectarse a estación do lugar onde están e alí engadir cancións para a súa reprodución. Normalmente estas plataformas contan co seu propio reprodutor de música e o seu propio catálogo, xeralmente bastante extenso pero non tanto coma calquera servizo de música xa que contan coas cancións mais actuais e os clásicos. Non existe un mercado moi amplo aínda que destacan algunhas aplicacións que se están comezando a implantar tanto nos Estados Unidos coma en Europa.

Comezando polas gramolas físicas temos a compañía **TouchTunes** está expandida por Norte América e parte de Europa dispoñible en 75.000 bares é a mais coñecida con diferenza, dispón dun gran catálogo de música, sistema de adaptación da música ao local e diferentes ferramentas para o entretemento dos clientes. Dispoñen de 3 tipos de gramolas, dúas de parede, unha que tan só sirve de reprodutor 3.1a e outra que contén mais funcionalidades coma o karaoke, unha última adaptable para axeitala o estilo do local coma por exemplo nun barril 3.1b. Todas funcionan con unha gran pantalla táctil, para facer a selección aínda que conta con unha aplicación móbil para este mesmo fin. Utiliza un sistema de créditos que se poden consumir nos diferentes establecementos nos que está implantada a plataforma, polo tanto o seu uso non é gratuíto. Páxina oficial TouchTunes

Figura 3.1: TouchTunes



(a) TouchTunes Virtuo (b) TouchTunes Fusion (c) Aplicación móbil

Como directa competidora de TouchTunes atopamos **AMI Entertainment**, comezou creando pianos nos que introducindo unha moeda podíase elixir algunha das cancións, posteriormente dedicaronse a facer gramolas físicas e nos nosos días seguen coa dixitalización destas.

Conta con uns 30.000 aparellos repartidos por todo o mundo. Ao igual que TouchTunes, tamén con 3 tipos de gramolas diferentes, moi similares as da súa máxima competidora pero inclúen unha de pé 3.2a, similar a unha das gramolas antigas. Tamén conta con aplicación móbil para engadir cancións, e tamén funciona cun sistema de créditos pra facer soar a música que mais che gusta. Páxina oficial de AMI entertainment

Figura 3.2: Ami Entertainment



(a) NGX Infinity

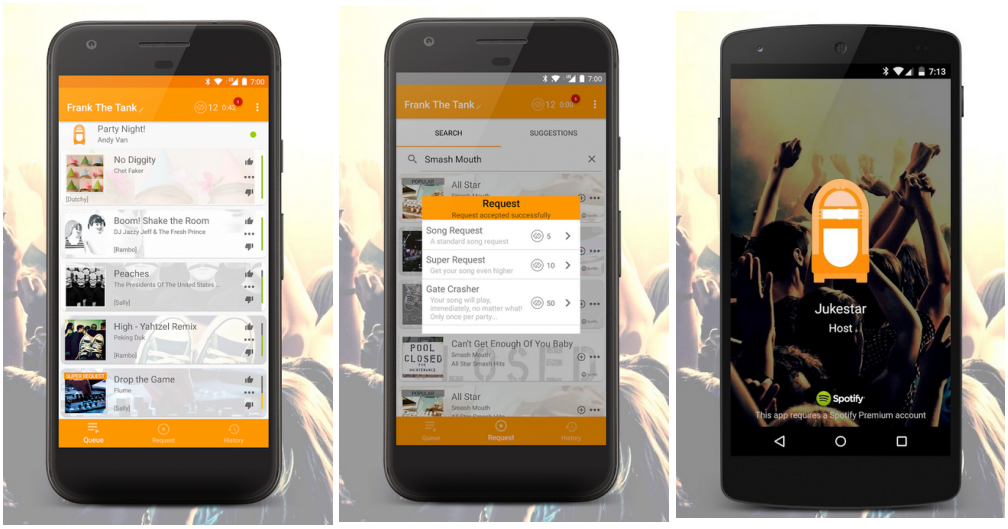


(b) Aplicación móbil

No apartado de aplicacións móbil que teñan unha funcionalidade similar a unha gramola atopamos **Jukestar** consiste en dúas aplicacións unha anfitrioa 3.3c e outra para os invitados 3.3a, o anfitrión ten que conectar a súa conta de Spotify na aplicación e xerar unha festa despois compartir a festa cos seus invitados, estes poden incluír cancións ou votar polas da súa preferencia, estas características funcionan cun sistema de créditos de pago 3.3b. É unha idea similar a da nosa aplicación, xa que utiliza Spotify para a reprodución de música pero necesita un cliente para o anfitrión e outro para os invitados o que complica o seu uso. Jukestar

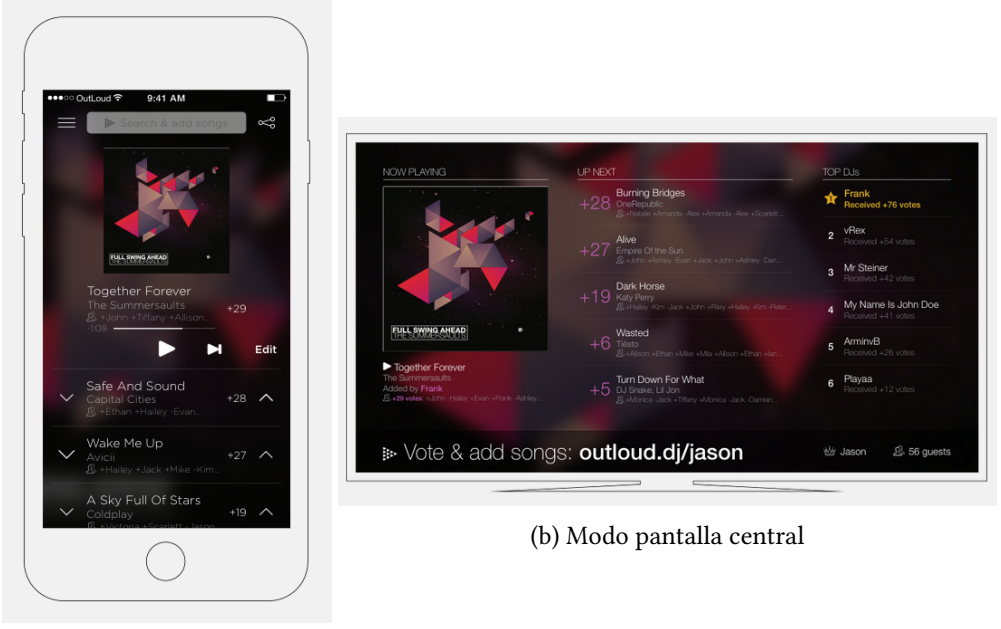
Outra aplicación similar é **Outloud** permite a reprodución de música dende Spotify, SoundCloud e incluso Apple Music para os trebellos compatibles. A idea e similar, créase unha playlist que faría de sesión e os usuarios poden conectarse xa sexa dende un enlace de invitación ou buscando por lugares próximos, despois poden engadir cancións e votar as que son mais do seu agrado, tamén permite o acceso dende a web sen necesidade de instalar a aplicación. Unha característica que inclúe e non existe noutras aplicacións é a posibilidade de proporcionar información da sesión nunha pantalla central 3.4b, nesta inclúense as seguintes cancións, as cancións recén engadidas e incluso quen son os usuarios que reciben mais votos nas súas cancións. Esta aplicación está pensada para usala en calquera lugar: vodas, ximnasios, lugar de traballo, festas particulares, reunións con amigos ... e pódese descargar en dispositivos Android ou iOS. Trátase dunha aplicación sinxela ademais de ser gratuíta, o único problema que podemos ver é que a aplicación necesita estar funcionando no dispositivo no que soa a música. Páxina oficial Outloud

Figura 3.3: Jukestar App



(a) Jukestar gramola (b) Solicitude de nova canción (c) Aplicación anfitrión

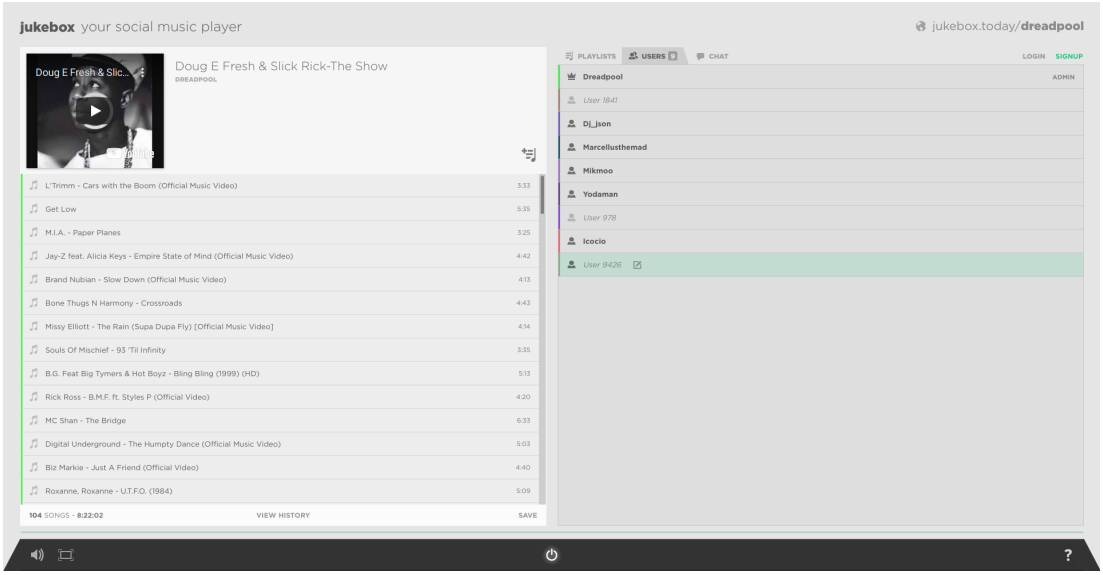
Figura 3.4: Outloud App



(a) Aplicación de Outloud (b) Modo pantalla central

Por outra banda, cambiando as aplicacións por servizos baseados na web, os que non necesitan aplicacións, existe **Jukebox Today** tratase dun servizo web que permite crear sesións e compartila cos usuarios. Permite que a música se reproduza de forma sincronizada en varios dispositivos ou controlar a reprodución dende calquera dos que estea activo, incluso conta con un sala de conversación, para os usuarios. Obtén a música dende YouTube polo tanto o seu catálogo é tan amplo como o da plataforma. Aínda que podería funcionar coma unha gramola non é a súa idea principal, xa que todos os usuarios poden facer modificacións na lista de reprodución ou poñer a pausa, trátase dun servizo gratuíto. Jukebox Today

Figura 3.5: Captura dunha sala en Jukebox Today



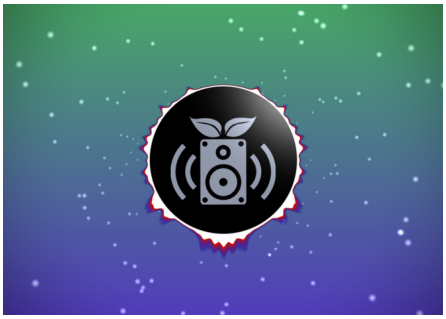
Con funcionamento completamente diferente **Raveberry** é un proxecto no que se instala nunha RaspberryPi un servidor musical multiusuario, é dicir, unha gramola. Contén unha interface intuitiva para a solicitude de cancións e cambiar a orde de acordo coa puntuación que os usuarios fan. Soporta as plataformas de Youtube, Spotify e SondCloud, incluso permite engadir arquivos locais. Está baseado en Python e contida no repositorio *PIP* e dispoñible tamén en *Docker* polo tanto é instalable en múltiples dispositivos aínda que é nunha RasberryPi onde se poden desfrutar a maioría das súas características que son moitas e moi ben pensadas. Citamos as mais destacadas que converten a Raveberry nun gran proxecto:

- **Streaming remoto:** permite reproducir música sincronizada noutros dispositivos.
- **Bluetooth:** pódese conectar a RaspberryPi a un altofalante bluetooth que permite evitar os cables.

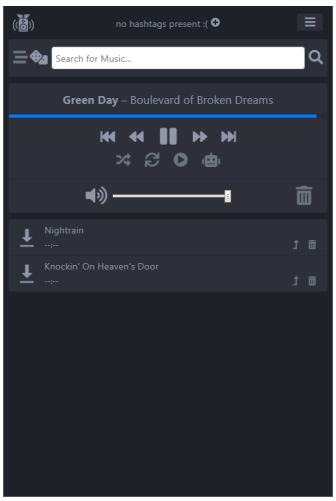
- **Hotspot:** creación dun punto WiFi no que os usuarios pódense conectar para usar a aplicación ou acceso a internet.
- **Roles de usuario:** permite engadir permisos a certos usuarios, por exemplo o control da reprodución.
- **Interface gráfica:** o administrador pode elixir a configuración desexada nunha cómoda interface.
- **Shareberry:** é unha aplicación complementaria que permite engadir cancións directamente dende o teléfono móbil.
- **Visualización de pantalla:** usando a aplicación cava, pódense ver as frecuencias de música nunha pantalla conectada 3.6a.
- **Visualización de audio:** pódese engadir dispositivos LED e Raveberry fará que funcionen ao ritmo da música.

Para o funcionamento de algunhas destas características é necesaria a inclusión de algún hardware adicional para o que o proxecto xa está preparado, por exemplo unha antena WiFi para a compartición da conexión, a integración dunha tarxeta de son para a mellora da calidade do son ou as luces LED que son controladas polos protocolos *i2c* e *spi*. Contén ademais, unha ampla documentación que fai que a súa instalación básica seña moi sinxela e trátase dun proxecto de código aberto polo tanto gratuíto. Repositorio de Raveberry en GitHub

Figura 3.6: Raveberry Server



(a) Visualización de pantalla



(b) Interface

Estes son os proxectos que atopamos cun funcionamento similar ao noso traballo, ningún destes cumpre todos os nosos requisitos aínda que varios aproximase a ideas similares, por

exemplo *Outloud* necesita unha aplicación aberta polo tanto no caso de esta estar pechada o servizo deixaría de funcionar, desta forma se dificulta o uso para o administrador. Destes traballos obtivemos ideas que poden ser implementadas no noso proxecto e poden engadir funcionalidades interesantes para o usuario final.

Capítulo 4

Planificación

Para a planificación dos proxectos se deben ter en conta tanto os custos como o tempo, neste capítulo explicase as etapas da desta durante o proceso de desenvolvemento, a planificación é acorde a metodoloxía Scrum 2.7.

4.1 Planificación inicial

Unha vez realizado a análise (capítulo 2) se decidiu dividir o traballo en 5 grupos:

- **Configuración inicial do proxecto:** Recolección de información sobre as ferramentas a utilizar para a posta en marcha do proxecto, configuración do entorno de desenvolvemento.
- **Implementación da persistencia:** Deseño da base de datos, modificación do modelo de usuarios de Django.
- **Conexión coa API de Spotify:** Creación da lóxica no backend, peticións necesarias da aplicación o API de Spotify. Configuración da xestión de usuarios.
- **Conexión frontend backend:** Planificación e implementación do frontend, conexión con JSON web Token (JWT) aos usuarios do backend.
- **Xeración do espazo web:** Peticións ao backend e xestión de permisos.

4.2 Adaptación da planificación a Scrum

O desenvolvemento do proxecto se dividiu en 4 fases, as cales tiveron unha duración de entre 3 e 5 semanas dependendo da complexidade, a metodoloxía foi adaptada a natureza do proxecto. Se realizou un Sprint Planning 2.7.2 ao principio de cada fase onde se obtivo o

Sprint Backlog 2.7.3 e ao finalizar cada unha se fixo o Sprint Review 2.7.2 e o Sprint Retrospective 2.7.2. Durante a realización do proxecto, tamén se levou un diario de traballo que facilitou a organización.

4.2.1 Primeiro sprint

No primeiro sprint se levou a cabo a implementación a primeira fase, se creou o Product Backlog 2.7.3 baseado nos requisitos funcionais. Posteriormente se estableceu o espazo de traballo, se configurou o editor NeoVim cunha selección de plugins adaptados ao proxecto, tamén se configurou o repositorio de GitLab. Neste sprint tamén incluímos as probas iniciais tanto coa API de Spotify coma con Ionic xa que se trataba dun framework novo para o desenvolvedor, por último tamén se realizaron os mockups do frontend 4.1.

4.2.2 Segundo sprint

Neste sprint se deseñou a base de datos, se crearon os modelos de datos en Django e se modificou o modelo xa preestablecido de usuarios en Django para engadir mais datos que este non contiña, na figura 5.2 podemos ver a súa estrutura.

4.2.3 Terceiro sprint

No terceiro sprint fíxose a conexión do backend coa API de Spotify, crearonse as peticións necesarias coa axuda da libraría de Python Spotipy, esta proporciona todas as funcionalidades necesarias, dende a obtención do token de autenticación contra a plataforma de música ata a reprodución, pasando pola xestión de playlists e a cola de reprodución.

4.2.4 Cuarto sprint

Para o cuarto sprint se planificou o deseño do frontend baseado nos mockups realizados. Tamén se fixo a conexión mediante JWT para a xestión de usuarios. Estudáronse outros métodos para facer dita conexión, pero se concluíu que a autenticación con JWT era a mais segura e útil no noso caso. Este sprint foi mais longo do esperado debido a novidade no uso deste tipo de autenticación, poñendo fincapé na especial dificultade de renovar os tokens de autenticación co token de refresco.

4.2.5 Quinto sprint

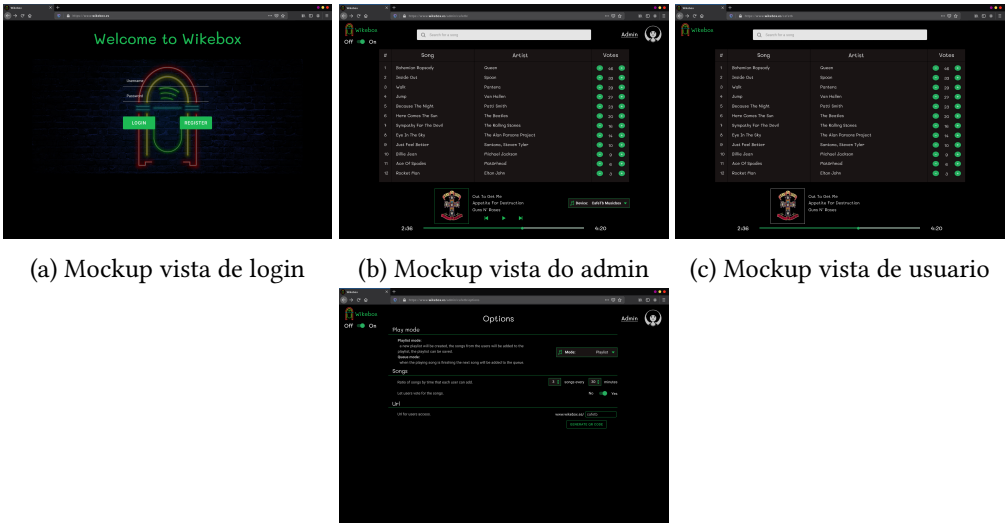
No quinto e último sprint se crearon as páxinas do frontend, se tivo en conta os permisos de acceso tanto na parte do frontend coma do backend permitindo tan só as accións necesarias.

4.3 Custos do proxecto

O software utilizado durante a creación deste proxecto foi sempre con licencias libres, en canto a recursos hardware non foron necesarios debido a que tan só foi necesario un ordenador do que xa se dispoñía. Polo tanto os recursos utilizados restrínxese tan só os recursos humanos.

Na táboa 4.1 mostrase os recursos humanos utilizados medidos en horas de traballo.

Figura 4.1: Mockups do frontend



(a) Mockup vista de login (b) Mockup vista do admin (c) Mockup vista de usuario

(d) Mockup opcións do admin

Táboa 4.1: Tempo empregado no proxecto

Tarefa	Tempo
Documentación	80h
Desenvolvemento do backend	100h
Desenvolvemento do frontend	200h
Probas	30h
Total	410h

Capítulo 5

Deseño

No seguinte capítulo explicáranse os aspectos relevantes do deseño da nosa aplicación, coma a arquitectura adoptada e os servizos utilizados.

5.1 Lóxica de funcionamento

Esta aplicación está deseñada coa idea de reproducir unha gramola das que existían habitualmente nos negocios de restauración pero de forma virtual, engadindo ademais algunhas características que creemos que poden ser útiles e facer o funcionamento desta máis atractivo para os usuarios finais. Coma é razoable non nos gustaría que a aplicación estivese deseñada nun nicho tan concreto coma o da restauración, polo tanto que sexa útil noutros ámbitos nos que se desexe crear unha lista de reprodución compartida sen complicación coma pode ser un ambiente de traballo cun fío musical, unha festa ou similares.
Ademais tamén sería desexable que a aplicación resultase sinxela para o usuario final, polo tanto para un novo usuario os pasos normais a seguir serían:

1. Creación dunha nova instancia ou sesión (poderíamos usar o símil dun bar).
2. Creación dun usuario nesta sesión, este ao ser o primeiro será o usuario administrador.
3. Conexión da sesión co usuario de Spotify.
4. Reproducir música en calquera dos clientes de Spotify, xa pode ser a web, os clientes de escritorio ou móbil.
5. Compartir o enlace da sesión para o resto dos usuarios.

Para o caso de novos usuarios nunha sesión xa rexistrada, poderíamos usar o símil de os clientes nun bar os pasos serían os seguintes:

1. Abrir o enlace recibido, que te achega a páxina de novo usuario dentro da sesión.

2. Rexistrar o novo usuario.

3. Engadir cancións e votar polas favoritas para que suban na cola de prioridade.

5.2 Arquitectura

No caso deste proxecto utilízanse dous frameworks diferentes no que cada seu teñen unha arquitectura diferente, Angular (o framework que usamos con Ionic) utiliza a típica modelo, vista, controlador e Django cambia esta derradeira por plantilla[1]. Ámbolos patróns de arquitecturas se basean nas ideas de reutilización do código e separación de conceptos o que separa as representacións internas da información co xeito que se presenta os usuarios finais da aplicación, estas características son básicas para facilitar a tarefa de desenvolver e facer mais doado o mantemento, pero o seu funcionamento non e de todo igual.

5.2.1 MVC: Modelo, vista, controlador

Trátase do patrón de deseño software máis coñecido, é moi utilizado no desenvolvemento web.

- **Model:** O **modelo** é a representación da información coa que o sistema opera, polo tanto xestiona todos os accesos a dita información, tanto consultas como actualizacións. Envía a vista a información que esta lle vai solicitando para ser mostrada na interface do usuario, as peticións de acceso ou manipulación chegan ao modelo a través do controlador.

- **View:** A **vista** encargase da representación de cara ao usuario de toda a información da aplicación, realiza as peticións ao modelo, necesarias para a recuperación de dita información. Proporciona unha experiencia interactiva do lado do cliente para o usuario, o que lle permite interactuar coa aplicación.

- **Controller:** O **controlador** é o encargado de interpretar as instrucións e accións que realizan os usuarios finais da aplicación, facendo uso das funcionalidades implementadas no modelo, validando os datos e os parámetros das solicitudes recibidas. Pódese dicir que o controlador fai de intermediario entre a vista e o modelo.

5.2.2 MVT: Modelo, vista, plantilla

A diferencia da arquitectura **MVC,** neste caso a parte do controlador é manexada directamente polo framework, é un patrón de deseño coñecido por Django.

[1] Template en inglés.

- **Model:** O **modelo** é similar ao caso de MVC, actúa como interface para os datos, é basicamente a estrutura lóxica de almacenamento de datos detrás da aplicación web ou a API no noso caso.
- **View** A **vista** executa a lóxica de negocio, fai iteración co modelo e renderiza a plantilla. Acepta peticións HTTP e responde tamén peticións HTTP. É similar ao controller explicada na arquitectura MVC.
- **Template:** A **plantilla** é o compoñente que marca a diferencia coa arquitectura MVC. A plantilla actúa como a capa de presentación, basicamente é o código HTML que renderiza a información. O contido das plantillas pode ser tanto estático coma dinámico, poderíase comparar a vista na arquitectura MVC.

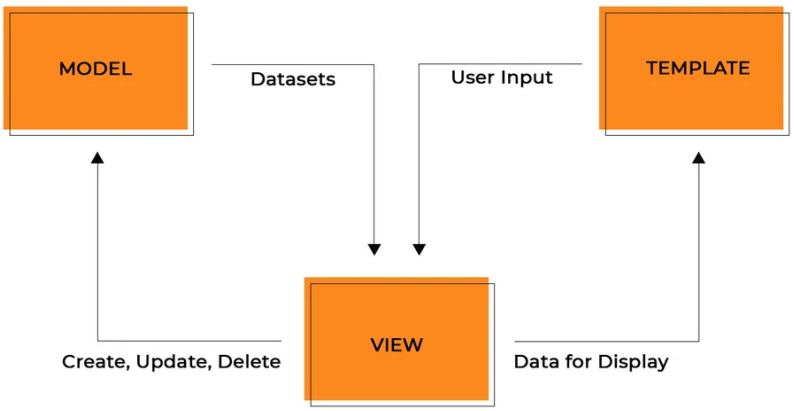


Figura 5.1: Model, Template, View.

Como vemos a gran vantaxe das arquitecturas é a separación das responsabilidades dentro da aplicación, deste xeito se existen cambios nos datos (Modelo) non afecta a visualización xa que esta se actualizará co contido novo, ademais de que o usuario non interactúa en ningún momento directamente cos datos.

5.2.3 Esquema xeral

O esquema xeral para o proxecto é o uso de Ionic para o Frontend e Django para o Backend, a elección destes Frameworks é: Ionic pola súa capacidade de desenvolver con tan só un código tanto unha aplicación web coma aplicacións específicas para os sistemas operativos mais usados nos móbiles hoxe en día, ademais do seu cliente que facilita a xestión de novos recursos ou o manexo destes. No caso de Django pola súa simpleza ao usar Python coma linguaxe de programación, o que simplifica o uso de librerías para a conexión con Spotify por exemplo, ademais de todas as características deste que facilitan o desenvolvemento do código

coma poden ser: o seu propio servidor para manexo do backend, que fai mais doadas as probas nos primeiros estadios da aplicación, ou o framework para as unit test de Python.

5.3 Frontend

O frontend en Ionic permite o uso de diferentes frameworks para o desenvolvemento, no noso caso utilizamos Angular xa que é ideal para a creación de Progressive web application (PWA) que é a principal idea no noso proxecto. Ademáis de que permítenos utilizar a programación reactiva, que a súa vez está enfocado no patrón de deseño observer que se basea na notificación duns obxectos a outros cando se producen cambios.

- **Programación reactiva:** trátase dun paradigma enfocado ao traballo con fluxos de datos, finitos ou infinitos de forma asíncrona. Ao ser asíncrona a recepción dos datos non fai bloqueos o que facilita a as chamadas a recursos lentos coma poden ser os que están en rede, chamadas a bases de datos, xestión de peticións e en xeral todo o fluxo de chamadas.

- **Observables:** é un modo de implementación da programación reactiva, reacciona ante o fluxo de distintos eventos. Consta de tres compoñentes principais: **Observable**, o que queremos observar, implementado mediante unha colección de eventos ou valores. **Observer**, é o actor que se dedica a observar, permite escoitar os eventos emitidos polos observables. **Subject**, o emisor de eventos, crea un fluxo cando un observable sufre cambios, estes eventos son consumidos polos observables.

5.3.1 Arquitectura

Coma indicamos anteriormente, Ionic segue o a arquitectura de MVC no noso caso tan só utilizamos o modelo para a comprobación dos JWT e para a transferencia de cancións dende o backend e recibir estas de forma correcta mediante un observable, para amosalo na vista. Na sección 5.4.1 explícase o modelo do backend que é o realmente utilizado na aplicación.

5.3.2 Vista

A vista é a última capa da arquitectura do sistema é a capa que interactúa co usuario mostrándolle a información e enviando as peticións do usuario ao controlador. Ao tratarse dunha aplicación web, a vista execútase nun navegador, ou nun dispositivo móbil coma PWA, polo tanto son páxinas HTML modificadas por JavaScript, codificado en TypeScript no noso caso grazas a Angular.

A continuación vaise expoñer brevemente a estrutura das vistas do frontend que se compón das páxinas en HTML:

- **Home:** trátase da páxina de destino para usuarios non conectados, nela explícase brevemente o funcionamento da aplicación e facilita o acceso as páxinas útiles para os usuarios. No caso dos usuarios xa conectados existe unha redirección a páxina de *Music* xa que se supón que os usuarios xa visitaron a páxina *Home*.
- **Instance:** nesta páxina os usuarios poden crear novas sesións para crear listas de reprodución conxuntas.
- **Login:** permite aos usuarios conectarse a súa sesión, existen páxinas de login para cada sesión, pero tamén é posible o acceso dende a páxina de login xenérica.
- **Music:** a páxina de music é a páxina principal da aplicación, nela os usuarios poden buscar cancións, engadilas a cola, visualizar a canción que está a soar no momento e votar as cancións para cambiar a súa orden. E no caso dos usuarios administradores, tamén poden seleccionar o dispositivo no que se reproduce a música e escoller unha lista de reprodución propia do seu Spotify para encher a aplicación e que os usuarios poidan votar a orde.
- **Register:** permite aos usuarios rexistrarse nunha sesión, cada sesión ten a súa páxina de rexistro, polo tanto os usuarios se rexistran directamente en dita sesión. Esta é a páxina que os usuarios administradores compartirán cos usuarios para permitirlles acceder a súa sesión.
- **Callback:** trátase da páxina a que redirixe Spotify despois de autenticarse na súa plataforma, esta captura o token do usuario.

5.3.3 Controller

O controller é a capa intermedia da arquitectura, actúa coma comunicación entre a vista e o modelo, no noso caso a maioría desta comunicación realizase no modelo, polo tanto moitas das instrucións na parte do controlador do frontend son peticións ao backend. Esta capa dividiuse en tres servizos que se describen a continuación:

- **Api:** este servizo componse de chamadas a API creada no backend, destaca a utilización de observables para a recepción dos datos poñemos de exemplo a función *queueSongs* 5.3.3 que recibe no observable a lista de cancións que se atopan nese momento na cola para ser votadas. Tamén cabe destacar a función *spotifyLoginUrl* 5.3.3 que abre unha nova pestana no navegador para que o usuario poida abrir a súa sesión en Spotify para

posteriormente capturar en *spotifyCallback* 5.3.3 a url de resposta e gardar o token. En tódalas funcións envíase a request xa que permite saber cal é o usuario que está facer a petición, este é o motivo de que as funcións sexan post e non haxa ningún get.

```
queueSongs(): Observable<IEsongs[]>{
  return this.httpClient.post<IEsongs[]>(this.SERVER_ADDRESS +
    'api/queuesongs/', {request: Request});
}
```

```
spotifyLoginUrl() {
  this.httpClient.post(this.SERVER_ADDRESS + 'api/token/',
    {'request': Request}).subscribe((response) => {
    window.open(response[0].url, '_self');
  })
}
```

```
  spotifyCallback() {
    this.httpClient.post(this.SERVER_ADDRESS + 'api/callback/',
    {'url': window.location.href })
  }
```

- **Auth:** o servizo de auth manexa a autenticación dos usuarios, usa JSON web Token para o control de acceso xa que permite a propagación de identidades e privilexios entre o provedor de información e o solicitante. Para o almacenamento dos JWT utilizase Ionic Storage xa usa o mellor método para o gardado de datos en cada plataforma utilizada. A función de *login* 5.3.3 é unha mostra do uso do almacenamento do token usando o storage.

```
login(username: String, password: String) {
  this.user = username
  return this.httpClient.post(this.SERVER_ADDRESS + 'users/token/',
    {
    "username": username,
    "password": password
  })
  .pipe(
    tap(token => {
      this.getInstance()
      this.token = token;
      this.storeJwtToken(this.token.access)
      this.storeRefreshToken(this.token.refresh)
      this.storeUser()
      this.storeExpiration()
      this.storage.set("token", this.token)
      .then(
```

```
        () => {
          console.log('Token Stored');
        },
        error => console.error('Error storing item', error)
      );
      return token;
    }),
  );
}
```

- **Interceptor:** para o uso de autenticación con JWT é necesario engadir o token en cada unha das peticións ao backend *intercept* 5.3.3 é a función que se encarga de interceptar a petición, a función *addToken* 5.3.3 é a encargada de engadir o token a cabeceira. Para garantir a seguridade os tokens de acceso caducan cada 5 minutos polo tanto necesitamos renovalos co token de refresco como podemos observar a función *handle401Error* 5.3.3 renova o token cando se produce un erro de autenticación.

```
intercept(request: HttpRequest<any>, next: HttpHandler):
    Observable<HttpEvent<any>> {

  if (this.authService.getJwtToken()) {
    request = this.addToken(request,
    this.authService.getJwtToken());
    console.log(request)
  }

  return next.handle(request).pipe(catchError(error => {
    if (error instanceof HttpErrorResponse && error.status === 401)
    {
      return this.handle401Error(request, next);
    } else {
      return throwError(error);
    }
  }));
}
```

```
private addToken(request: HttpRequest<any>, token: string) {
  return request.clone({
    setHeaders: {
      'Authorization': `Bearer \${token}`
    }
  });
}
```

```
private handle401Error(request: HttpRequest<any>, next:
    HttpHandler) {
  if (!this.isRefreshing) {
    this.isRefreshing = true;
    this.refreshTokenSubject.next(null);

    return this.authService.refreshToken().pipe(
      switchMap((token: any) => {
        this.isRefreshing = false;
        this.refreshTokenSubject.next(token.access);
        return next.handle(this.addToken(request, token.access));
      }));

  } else {
    return this.refreshTokenSubject.pipe(
      filter(token => token != null),
        take(1),
      switchMap(access => {
        return next.handle(this.addToken(request, access));
      }));
  }
}
```

5.4 Backend

Aínda que foi deseñado coa idea de publicar noticias dun xornal, Django é un dos frameworks mais populares hoxe en día, segue o principio (DRY, Don't Repeat Yourself) [2] polo que evita a repetición de código innecesariamente. Está escrito en Python, que axuda a súa popularidade, ademais como xa explicamos utiliza o a arquitectura **MVT** que destaca pola simpleza a hora de cambiar a forma de visualizar o contido.

5.4.1 Modelo

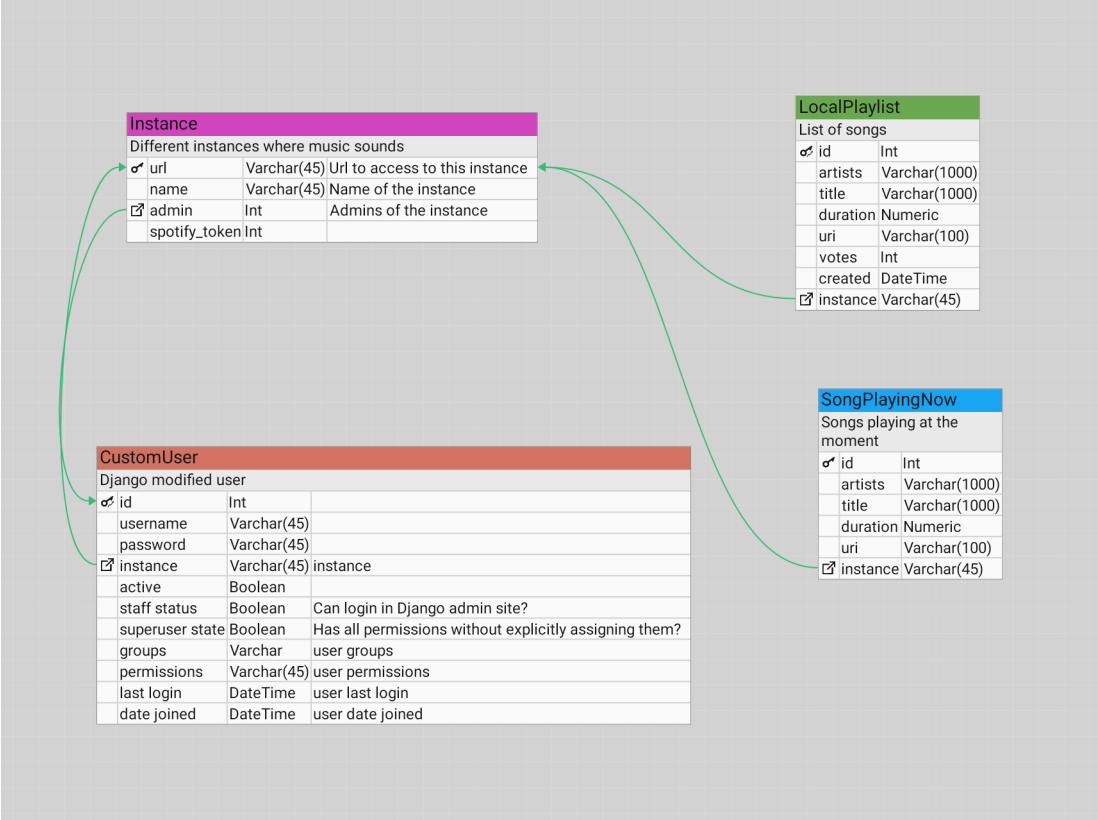
A capa modelo é a parte da aplicación que encapsula a lóxica de negocio cos accesos a base de datos. Nesta capa se almacea, modifica e eliminan os datos que manexa a aplicación. Na figura 5.2 pódense ver as entidades do modelo. A continuación detallase brevemente cada unha das entidades.

- **Instance:** representa os datos das sesións, almacenase a lista de sesións que estean rexistradas na aplicación.

[2] Non te repitas

- **LocalPlaylist:** representa a lista de cancións na cola das sesións.
- **SongPlayingNow:** representa a canción que se está a reproducir nas sesións.
- **CustomUser:** representa os usuarios da aplicación, é unha modificación do modelo User de Django adaptado para o proxecto

Figura 5.2: Estrutura da base de datos



5.4.2 Plantilla

Debido a que utilizamos Django coma backend non é necesario o uso da plantilla para amosar os datos, esta responsabilidade recae no Frontend.

5.4.3 Vista

Na arquitectura MVT é a vista a encargada de executar a lóxica de negocio. Recibe as peticións dende o arquivo *urls.py* e executa as accións necesarias, no noso caso dividimos as peticións entre as necesarias para administrar os usuarios e as relacionadas coa música, dividindo esta última de novo entre as peticións a nosa API e as da API de Spotify.

User views:

debido a que utilizamos o modelo de usuario de Django modificado é este o encargado do manexo de case todas as peticións, nos tan só tivemos que engadir a obtención da sesión do usuario coma podemos ver en 5.4.3.

```
1 @api_view(['POST'])
2 def getInstance(request):
3     user =
    CustomUser.objects.filter(username=request.data['username']).values()
4     userInstance = user[0]['userInstance_id']
5     return Response(data=userInstance, status=200)
```

Api views:

as funcións explicadas a continuación son algunhas das mais destacables das solicitudes que se fan a xestión da nosa API, todas as peticións son de tipo POST debido a que sempre temos que saber cal é o usuario que fai a solicitude.

- **PopTop** é a función encargada de borrar a primeira canción da lista se é a que está a soar.

```
1  def popTop(wike):
2      current =
       SongPlayingNow.objects.filter(instance=wike).values('uri').first()
3      top =
       LocalPlaylist.objects.filter(instance=wike).order_by('-votes',
       '-added_mode', 'created').values('uri').first()
4      if current != None:
5      if current == top:
6          res = "same"
7
       LocalPlaylist.objects.filter(instance=wike).order_by('-votes',
       '-added_mode', 'created').first().delete()
8      else:
9          res = "notSame"
10     else:
11         res = "Not playing"
12     return res
```

- **AddToQueue** engade a seguinte canción a cola de reprodución de Spotify, comproba que falten menos de 30 segundos da canción que está a soar e engade a seguinte canción da lista local de cancións. Utiliza unha variable global xa que é necesario saber cal foi a última canción engadida para non engadila mais dunha vez.

```
lastInQueue = None # Global var for keep the last song added to the queue
def addToQueue(wike):
    totalTime = SongPlayingNow.objects.filter(instance=wike).values('duration').first() # Get the total duration of the song
    nowTime = spotify.currentProgress(wike) # Get the progress of the song
    if (totalTime != None) and (nowTime != None): # Check that there are a song playing
        totalTime = totalTime['duration']
        nowTime = timedelta(milliseconds=nowTime['progress_ms'])
        changeTime = nowTime + timedelta(seconds=30) # Establish the time to add the next song
        if changeTime > totalTime:
            adding = LocalPlaylist.objects.filter(instance=wike).order_by('-votes', '-added_mode', 'created').values('uri').first()
            global lastInQueue
            if adding != lastInQueue: # Check if the last added song is the same as the one at the top of the LocalPlaylist
                spotify.addToQueue(wike)
                lastInQueue = LocalPlaylist.objects.filter(instance=wike).order_by('-votes', '-added_mode', 'created').values('uri').first()
                res = "Change"
            else:
                res = "Already added"
        else:
            res = "No change"
    else:
        res = "No playing"

    return res
```

- **VoteP** é a función de votación, permite ao usuario engadir un voto novo a canción elixida.

```
@api_view(['POST'])
@permission_classes([IsAuthenticated])
def voteP(request):
    N = LocalPlaylist.objects.values_list('votes', flat=True).get(instance=request.user.userInstance, uri=request.data['uri'])
    N = N+1
```

```
6
    LocalPlaylist.objects.filter(instance=request.user.userInstance).filter(uri=request.d
7   return Response(data="plus", status=200)
```

- **CheckerThread** para implementar a funcionalidade de que o administrador non teña que ter a páxina da aplicación aberta e as comprobacións para detectar se a canción vaise acabar ou se ten que borrar a seguinte, creamos unha función que executa ditas comprobacións cada 10 segundos. Esta función execútase nun fío paralelo, xa que Django ten que poder responder ao resto das peticións e desta forma evitamos o bloqueo.

```
1  def checkerThread():
2      while True:
3          wikes = instance.objects.all()
4          for wike in wikes:
5              if spotify.areToken(wike):
6                  if spotify.currentProgress(wike) != None and
       spotify.currentProgress(wike)['is_playing']:
7                      addToQueue(wike)
8                      popTop(wike)
9          time.sleep(10)
10
11 x = threading.Thread(target=checkerThread)
12 x.start()
```

Spotify views:

o conxunto seguinte de funcións son as relacionadas coas peticións a API de Spotify, algunhas son simplemente a petición directamente, por isto só amosamos as mais destacables.

- **Amanager** esta función é a encargada de xerar o xestor de autenticación de Spotify.

```
1  def amanager(userInstance):
2      instance = str(userInstance)
3      auth_manager = spotipy.oauth2.SpotifyOAuth(client_id=client_id,
       client_secret=client_secret, redirect_uri=redirect_uri,
       scope=scope, cache_path=".spotify_token_" + instance)
4      return auth_manager
```

- **ChooseDevice** permite ao administrador elixir en que dispositivo quere que se reproduza a música no caso de que haxa mais de un.

```
1  def chooseDevice(userInstance):
2      client = getClient(userInstance)
3      devices = client.devices()
```

```
    devices = devices['devices']
    devicesDict = {}
    # If no active devices send no active devices
    if len(devices) >= 1:
        for i in range(len(devices)):
            devicesDict[i] = {
                    'id': devices[i]['id'],
                    'name': devices[i]['name']
                    }
    else:
        devicesDict[0] = {
                'id': "No id",
                'name': "No active devices"
                }

    devicesJson = json.dumps(devicesDict)
    return devicesJson
```

- **AddToQueue** engade a seguinte canción a cola de reprodución de Spotify.

```
def addToQueue(userInstance):
    client = getClient(userInstance)
    curi =
    LocalPlaylist.objects.filter(instance=userInstance).order_by('-added_mode',
    '-votes', 'created').values('uri').first() # Get the uri of the
    next song
    ccuri = json.dumps(curi['uri'])
    cccuri = ccuri[15:]
    rcccuri = cccuri.replace("\"", "")
    try:
        client.add_to_queue(rcccuri, None)
    except:
        pass
```

- **Search** fai a procura das cancións para o usuario.

```
    def search(search, userInstance):
        client = getClient(userInstance)
        searchResponse = client.search(search, limit=10) # Retuns a
    'dict' of songs
        cleanlist = searchResponse['tracks']['items'] # Gets the
    important part of the response
        # Creates a dictionary and populates it with the responses
        songList = {}
        for i in range(len(cleanlist)):
            songList[i] = {
```

```
                    'song': cleanlist[i]['name'],
                    'artists': cleanlist[i]['artists'][0]['name'],
                    'uri': cleanlist[i]['uri']
                    }
        songJson = json.dumps(songList) # Convert the dictionary to
    a json
        return songJson
```

- **SearchToLocalPlaylist** engade a canción seleccionada ao modelo de datos.

```
def searchToLocalPlaylist(songUri, userInstance):
    client = getClient(userInstance)
    playlist = LocalPlaylist()  # Assign the model
    searchedSong = client.track(songUri)
    playlist.pk = None  # The pk is assigned automatically, this is
    needed for that
    artists = searchedSong['artists']  # Loop in the artist if
    there are more than one
    string = ""
    for j in range(len(artists)):
        string += artists[j]['name']
        if j+1 != len(artists):
            string += ", "  # Separate the artist names
    playlist.artists = string
    playlist.title = searchedSong['name']
    playlist.duration = datetime.timedelta(milliseconds =
    searchedSong['duration_ms'])
    playlist.uri = searchedSong['uri']
    playlist.created = datetime.datetime.now()
    playlist.added_mode = "search"
    playlist.instance = userInstance  # The instance of the user
    comes as parameter
    playlist.save()
    return json.dumps(searchedSong)
```

Capítulo 6

Conclusións e traballo futuro

Derradeiro capítulo da memoria, onde se presentará a situación final do traballo, as leccións aprendidas, a relación coas competencias da titulación en xeral e a mención en particular, posibles liñas futuras.

6.1 Conclusións

O sistema creado cumpre os requisitos desexados, creamos unha gramola virtual que permite aos usuarios o uso mais básico de estas, que é a reprodución das cancións preferidas nunha orde. Ademais de ser un sistema o mais sinxelo posible o que facilita o uso xa que con poucas iteracións do usuario pódese crear unha sesión e reproducir música en calquera dispositivo con Spotify de forma colaborativa.

6.2 Traballo futuro

Coma calquera proxecto software, sempre cabe a posibilidade da implementación de novas ideas que melloren o produto final, no noso caso a lista podería ser moi longa xa que existen moitos sistemas diferentes para a reprodución de música e a inclusión de algúns destes melloraría o valor da nosa aplicación. Nun principio a aplicación non busca o lucro, aínda que se poderían poñer en funcionamento algunhas funcionalidades para que así fose. A continuación expoñemos algunhas das ideas que se poderían desenvolver para a mellora da aplicación, estas son sempre opcións non exclusivas, polo que cun panel de axustes o administrador podería elixir a súa configuración desexada.

- **Modo DJ:** cun funcionamento similar, a aplicación non engadiría as cancións automaticamente a cola, mostraría unha lista a unha persoa que se dedique a pinchar segundo os votos dos usuarios e as súa propia elección.

- **Sistema de créditos:** limitar o número de cancións por usuario e ofrecer un código para conseguir aumentar o límite, por exemplo nun negocio de restauración para incitar o consumo dos clientes.
- **Usuarios globais:** poder ter o mesmo usuario para diferentes sesións en diferentes lugares. Deste xeito poderíase manter un avatar e unhas estadísticas de uso persoais.
- **Estadísticas:** amosar aos usuarios estadísticas da sesión, cancións mais reproducidas, usuarios que mais cancións engaden...
- **Gardar listas:** as cancións que soen nun período de tempo gardalas nunha lista de reprodución de Spotify, deste xeito poderíase gardar por exemplo a lista de reprodución dunha noite nun bar.
- **Escoller estilos musicais:**

Apéndices

Bibliografía